From Semantic Safety to Candidate-Rule-Space Reduction in Snort-to-P4 Compilation

Arthur de Freitas Medeiros
Universidade Federal do Pampa (UNIPAMPA)
arthurfreitas.aluno@unipampa.edu.br

September 2026

Abstract

Programmable data planes can evaluate lightweight packet predicates before a software intrusion detection system (IDS), but Snort 3 rules also depend on payload, stream, flow/state, and other semantics that a P4 target cannot necessarily represent. We present a semantics-aware, target-aware Snort-to-P4 compilation approach that classifies each rule-to-predicate relation as Exact, Necessary, or Unsupported. A necessary predicate is meant to retain every packet satisfying its source rule within an explicitly enforced packet domain, while omitted semantics remain explicit in a manifest recording predicate-to-rule provenance. Packets outside that domain, such as those carrying IPv4 options, fragments, or truncated headers, are failed open as conservative candidates instead of being matched against fields the parser cannot locate. Among 4,017 evaluated rules, 4,003 were classified as necessary, 14 as unsupported, and none as exact; backend lowering produced 331 final P4 installation entries. Despite this semantic coverage, the global binary prefilter marked 100% of parser-eligible packets as candidates across SQLi, XSS, and held-out Brute Force traces, an outcome that causal analysis attributes to both broad necessary predicates and distributed union coverage. Provenance nevertheless supports an offline utility, deriving per-packet candidate rule sets: on held-out Brute Force, the average set decreased from the frozen 3,611-rule P4-eligible partition to 324.52 rules, a 91.01% reduction, or 81.81% over the full ruleset once the 406 fallback rules that apply to every packet are counted. Observed alerts were preserved in the evaluated traces, and in an end-to-end campaign over 1,000 sampled rules, with Snort 3 as the oracle and BMv2 as the target, no rule that a plan prefilters alerted on a witness the data plane marked no-match. We also show that the necessary relation holds only where Snort evaluates the full rule header. Once Snort 3 identifies a flow's service, it ignores the header ports and protocol of service-based rules, and 2,928 necessary rules are service-based with predicates that constrain the protocol and, for 2,877, a port. The global plan still covers all but nine of them through its port-free entries. The strong-discriminator ablation behind the candidate sets does not,

and recompiled with Snort 3's semantics it keeps 739 rules eligible, which bounds the full-ruleset candidate reduction at 18.4%. These results show that semantic safety and operational selectivity are distinct, and that what a rule-derived predicate guarantees depends on how the IDS evaluates the rule; candidate-set-aware IDS execution remains future work.

1 Introduction

Signature-based network IDSs remain dependent on rules that combine packet headers with payload inspection, stream and flow state, service semantics, and temporal conditions. These conditions cannot generally all be represented by a programmable data-plane target. The resulting challenge is not merely to translate rule syntax to P4, but to partially compile Snort 3 detection semantics without obscuring what the target cannot represent.

Programmable data planes can evaluate lightweight packet predicates before software IDS processing. Prior P4-based IDS and prefiltering efforts motivate this form of data-plane assistance [7, 14], while work on rule-based prefiltering emphasizes the importance of retaining the context required by the IDS [3]. However, the interaction between semantic preservation, target limitations, and ruleset-scale selectivity remains insufficiently characterized.

A Snort rule may contain representable header predicates together with payload, stream, flow/state, normalization, or service-dependent conditions. Compiling only the representable subset without recording the omitted semantics risks presenting a partial implementation as equivalent to the original rule. A safe necessary prefilter instead establishes, for a rule r and generated predicate p_r, the relation $r \Rightarrow p_r$ under a stated semantic profile and a stated packet domain; it does not assume $p_r \Rightarrow r$ or $r \Leftrightarrow p_r$. Traffic the data plane cannot parse as intended is retained by failing open rather than being matched against fields read from the wrong offset. The detailed model is developed in the problem formulation.

Semantic safety alone, however, does not determine operational usefulness. When thousands of necessary predicates are aggregated into a global binary candidate decision, their union can cover nearly all traffic in a workload. Semantic coverage and packet-level selectivity must therefore be evaluated as separate properties. Moreover, the binary decision discards which predicate and which source rule produced a match. Retaining predicate provenance enables an alternative question: not only whether a packet remains a candidate for IDS handling, but which IDS rules remain plausible candidates for that packet. This is evaluated here as an offline candidate-rule-space opportunity, not as an implemented runtime execution mechanism.

This paper develops and evaluates a semantics-aware, target-aware compilation workflow from Snort 3 rules to P4 predicates, with explicit delegation and fallback. Under the evaluated configuration, 4,003 of 4,017 rules admitted necessary predicates, although for the 2,928 service-based ones the relation holds only where Snort evaluates the full header, and the global binary prefilter marked

100% of parser-eligible packets as candidates in the three evaluated traces. In contrast, provenance-based analysis retained a 91.01% mean candidate-rule-space reduction on held-out Brute Force within the frozen 3,611-rule P4-eligible partition, or 81.81% over the full ruleset; with Snort 3's service semantics, the same derivation could save at most 18.40% of the full ruleset. These observations motivate the following contributions and research questions.

1.1 Contributions

This work makes the following contributions:

- A semantics-aware, target-aware method for deriving P4-executable predicates from Snort 3 rules and classifying their relation as EXACT, NECESSARY, or UNSUPPORTED.
- A compilation manifest that records preserved and delegated predicates, fallback and unsupported rules, and provenance linking final P4 entries to source Snort rule identifiers.
- A P4 backend that performs predicate expansion, normalization, target-aware lowering, deduplication, and P4Runtime installation-plan generation; the evaluated backend reduces 6,675 predicate instances to 331 final installation entries as installation state.
- An explicitly enforced packet domain that gates predicate evaluation on IPv4 header form and fails open for options, fragments, truncated headers, and parser errors, validated by a controlled data-plane suite and by trace-level replay accounting.
- An empirical evaluation that separates semantic safety from ruleset-scale packet-level selectivity, including a causal analysis of broad predicates and distributed union coverage.
- A provenance-based, offline candidate-rule-space analysis that retains substantially smaller per-packet rule sets, including a 91.01% mean reduction on held-out Brute Force within the 3,611-rule P4-eligible partition and 81.81% over the full ruleset, bounded at 18.40% once service-based rules are compiled as Snort 3 evaluates them.
- An analysis, from the Snort 3.12.2.0 source, showing that rules with a `service` option do not make their header ports and protocol necessary conditions, with a measurement of the 2,928 affected rules, of which plans remain candidate-complete for them, and of what a compilation faithful to that rule changes.
- Empirical validation of semantic-split behavior and observed P4 alert-packet preservation, without claiming formal equivalence or dynamic stateful candidate-set execution.

- An end-to-end differential campaign that synthesizes a witness flow per rule and compares Snort 3 alerts with BMv2 verdicts frame by frame, observing no violation over a stratified sample of 1,000 rules and measuring how much the evaluated plans can falsify.

1.2 Research Questions

RQ1 – Semantic Coverage. To what extent can Snort 3 rules be safely represented by predicates executable in a programmable P4 data plane while explicitly preserving semantic limitations?

RQ2 – Backend Feasibility. Can the semantically derived predicates be lowered and installed in a practical P4 target without hiding unsupported semantics or exceeding target constraints?

RQ3 – Operational Selectivity. How selective is a binary P4 prefilter when thousands of semantically safe predicates are jointly deployed?

RQ4 – Detection Preservation and Alternative Utility. When packet-level bypass selectivity is limited, can the compiled predicates still preserve observed IDS alerts and reduce the per-packet candidate rule space?

1.3 Paper Organization

The remainder of the paper presents background and related work, followed by the problem formulation and proposed approach. It then describes the experimental methodology, results, discussion, threats to validity and limitations, conclusion, and the experimental-profile appendix.

2 Background

2.1 Snort 3 Rule Semantics

Snort [11] is a signature-based network IDS; this work targets Snort 3 [17]. A Snort 3 text rule has a header and a body. The header names an action, a protocol, the source address and port, a direction that is either unidirectional or bidirectional (`<>`), and the destination address and port; addresses and ports may be variables such as `$HOME_NET` or the wildcard `any`. The body, in parentheses, holds options: payload options such as `content` and `pcre` that inspect a buffer, flow options such as `flow` and `flowbits` that depend on session state, and metadata such as `sid`. Snort selects candidate rules with a fast-pattern search and then evaluates each candidate's options left to right, stopping at the first one that fails.

Snort 3 also distinguishes port-based from service-based rules. A rule is service-based when it declares a `service` option, uses an option specific to a service, or names a service protocol in its header, as in `alert http`. Every header component of a port-based rule must match. Once Snort has identified the flow's service as one of the services of a rule with a `service` option, it checks the rule's header addresses but not its ports or protocol; a rule that names

the service protocol in its header keeps its port checks (Section 4.5). A header predicate taken from a rule is therefore an approximation whose relation to the rule has to be stated rather than assumed.

2.2 P4 Programmable Data Planes

P4 [2] programs describe how a data plane parses packets and processes them through match-action tables; this work uses $P4_{16}$ [15]. A parser extracts headers into typed fields, and control blocks apply tables whose keys use match kinds such as `exact`, `ternary`, and `lpm`; the v1model architecture adds `range` and `optional`. The program fixes which fields a table can match, while the control plane installs the entries at run time. The evaluated target is BMv2 [10], whose `simple_switch_grpc` process implements the v1model architecture and serves the P4Runtime API [16]. Through it a controller installs the pipeline configuration and writes table entries, and ternary entries carry a priority that orders overlapping matches. The evaluated backend uses target-supported Ethernet/IPv4 header fields as a candidate marker; it does not replace Snort detection.

2.3 Semantic Preservation and Prefiltering

Necessary predicates can safely retain traffic for Snort inspection, but their joint deployment need not provide useful packet-bypass selectivity.

3 Related Work

3.1 Rule-Derived and Hybrid P4 IDS Offloading

Rule-derived filtering has established that programmable data planes can assist software IDS processing. P4ID [7] derives packet filters from Snort rules using information such as IP addresses, transport-layer ports, protocols, and TCP flags, and combines stateless and stateful packet-processing stages. It therefore demonstrates both rule-derived filtering and stateful data-plane assistance. P4-ONIDS [14] addresses constrained data planes through range representation, rule aggregation, deduplication, and flow information to limit packets forwarded to the NIDS.

4MIDable [8] provides a framework for integrating existing security middleboxes with P4/SDN infrastructure. Its P4ID agent supports stateful processing and feedback to an unmodified IDS, and was evaluated on both software BMv2 and hardware Tofino targets. The work reports traffic-processing reductions without affecting observed detection rates. P4NIPS [12] provides a recent P4 network-intrusion prevention implementation. Together, these systems show the feasibility of rule-derived filtering, hybrid IDS/data-plane integration, stateful assistance, and resource-aware P4 representations; they are not treated here as evidence for a common per-rule semantic relation between a source IDS rule and a generated predicate.

3.2 Context Preservation and Hybrid IDS Processing

Brum et al. [3] revisit rule-based NIDS pre-filtering by considering traffic required for subsequent IDS processing even when a packet does not directly satisfy extracted rule conditions. Their eRBF approach incorporates flow context, including TCP connection establishment and termination information and selected protocol dependencies. Evaluated with Snort 3 and large rulesets, it shows why aggressive pre-filtering must account for context required by the IDS. Its software trace-based evaluation, however, does not address the mapping of those semantics to a P4 target's processing and resource constraints.

RNA [5] follows a complementary event-oriented direction: it analyzes selected Zeek-script events and maps them to protocol and event templates for programmable forwarding devices, together with complementary host-side mechanisms. This coordinated generation illustrates how selected IDS functionality can be distributed across host and data-plane components. Its scope is selected Zeek events and predefined templates rather than a general semantic classification of signature-based IDS rules according to the guarantees preserved by their data-plane representation.

3.3 Programmable-Data-Plane Security Landscape

The broader programmable-data-plane literature situates IDS offloading among other security applications and target models. Surveys of P4 fundamentals and applications [6], P4 security applications and their vulnerability context [1], and programmable-data-plane IDS approaches [4] document the breadth of the field. Implementation-oriented work also emphasizes that mapping security functionality to programmable targets involves resource and implementation constraints [9]. These sources motivate target-aware treatment of security functionality; they do not by themselves establish properties of the individual systems compared above.

3.4 Positioning of This Work

Prior systems demonstrate that rule-derived filtering, stateful assistance, resource-aware P4 representations, and coordinated IDS/data-plane execution are feasible. This work focuses on making the semantic consequences of partial compilation explicit. It classifies the relation between each source Snort rule and generated P4 predicate, records preservation, delegation, and fallback in a compilation manifest, and retains source-rule provenance through backend lowering and deduplication.

The evaluation also separates semantic safety from aggregate operational selectivity: a safe necessary predicate can be retained for Snort inspection yet contribute to a global binary candidate union that is not selective. Rather than presenting that outcome as a successful traffic-bypass result, the analysis uses retained provenance to examine an offline per-packet candidate-rule-space opportunity. Thus, the contribution is not Snort-to-P4 translation alone, but explicit

semantic and target-aware characterization of what is preserved, delegated, and still operationally useful. Table 1 compares these dimensions across the reviewed work.

Table 1: Related-work comparison. "Not explicit" denotes information not established in the reviewed scope, rather than absence of the feature.

Work	**Source IDS / rules**	**P4 target**	**Rule-derived filtering**	**State / context handling**	**Explicit semantic relation**	**Provenance retained**	**Main objective**
P4ID	Snort rules	P4 data plane	Yes	Yes (stateful stage)	Not explicit in reviewed scope	Not explicit in reviewed scope	Rule-derived IDS filtering
P4-ONIDS	NIDS rules	P4 data plane	Yes	Partial (flow information)	Not explicit in reviewed scope	Not explicit in reviewed scope	Resource-aware NIDS pre-filtering
4MIDable	Existing security middleboxes / P4ID agent	BMv2; Tofino	Yes	Yes	Not explicit in reviewed scope	Not explicit in reviewed scope	Hybrid security-VNF offloading
eRBF	Snort 3 rules	N/A (software trace model)	Yes	Yes	Not explicit in reviewed scope	Not explicit in reviewed scope	Context-preserving NIDS pre-filtering
RNA	Zeek events	P4 device + host	N/A	Partial (event / host coordination)	Not explicit in reviewed scope	Not explicit in reviewed scope	Event offload
P4NIPS	Not explicit in reviewed scope	P4	Not explicit in reviewed scope	Not explicit in reviewed scope	Not explicit in reviewed scope	Not explicit in reviewed scope	NIPS implementation
This work	Snort 3 rules	BMv2	Yes	Delegated to Snort	Yes (three classes)	Yes	Semantic transparency and selectivity analysis

4 Problem Formulation

4.1 Snort Rule Model

A signature-based IDS rule can combine multiple conditions that depend on different types of information. Some conditions can be evaluated directly from packet headers, while others may depend on payload contents, flow state, temporal relationships, protocol normalization, or information collected across multiple packets. Therefore, a rule cannot generally be treated as a single match-action entry when considering its execution in a programmable data plane.

For the purposes of this work, a rule r is represented as:

$$r = (H_r, P_r, F_r, T_r, N_r, a_r, m_r),$$

where H_r represents header predicates, P_r payload-dependent predicates, F_r flow and state conditions, T_r temporal conditions, N_r normalization or protocol-dependent conditions, a_r the associated action, and m_r rule metadata such as its identifier and priority. Given a target data plane s, only a subset of the predicates that compose a rule may be executable by that target. We denote by C_s the set of capabilities exposed by the target, such as exact, ternary, or range matching over selected packet-header fields. The compilation process analyzes each rule predicate with respect to C_s and produces a data-plane predicate p_r together with a semantic classification.

4.2 P4 Predicate Model and Semantic Relations

In this initial model, the generated representation is assigned to one of three classes:

- **EXACT**: all predicates considered by the supported rule profile are represented without semantic relaxation in the generated data-plane predicate.
- **NECESSARY**: the generated predicate preserves only conditions that are necessary for the original rule to match. Therefore, packets that satisfy the generated predicate still require inspection by the original IDS.
- **UNSUPPORTED**: the compiler cannot derive a safe data-plane predicate under the capabilities assumed for the target, and evaluation remains delegated to the IDS.

4.3 Safety Condition

The semantic classification determines how the generated component can be interpreted. An EXACT representation records no delegated, fallback, or unassessed detection predicate within the supported profile. The current P4 program nevertheless implements candidate marking through the single `send_to_ids` action rather than a general data-plane detection action. A NECESSARY representation is used only as a conservative pre-filter: matching traffic remains subject

to inspection by Snort, since the data-plane predicate alone is insufficient to reproduce the complete detection decision. An UNSUPPORTED rule remains delegated to Snort. Therefore, the existence of a data-plane match is not treated as equivalent to an IDS alert unless the required semantic guarantees are satisfied. For a rule r and a generated NECESSARY predicate p_r, the safety relation is $r \Rightarrow p_r$; the converse $p_r \Rightarrow r$ is not assumed. UNSUPPORTED means that no safe selective P4 predicate was established under the target profile, not that the rule is ignored.

4.4 Predicate-Level Soundness of Partial Compilation

The NECESSARY claim is a predicate-level statement under the current Ethernet/IPv4 target profile, not a general claim about arbitrary Snort execution. For a parsed rule r, let H_r denote the conjunction of its header conditions that the capability analysis marks PRESERVED or EXPANDED; let U_r denote the remaining detection-relevant conditions recorded as delegated, fallback, or unassessed. For a rule whose header Snort evaluates in full, a source-rule match entails its represented header conditions, so $r(x) \Rightarrow H_r(x)$; Section 4.5 treats the rules for which Snort does not. The backend materializes a predicate $p_r = L(H_r)$ only for a prefilter candidate. Under the implementation assumptions that the accepted header forms are parsed and normalized as recorded in the manifest, and that lowering does not exclude a packet satisfying H_r, the construction gives

$$r(x) \Rightarrow H_r(x) \Rightarrow L(H_r)(x) = p_r(x).$$

The implication uses no claim about the omitted U_r conditions: their presence can make p_r broader, which is why NECESSARY is not EXACT. If the compiler cannot establish a selective H_r, it emits no prefilter entry and classifies the rule as UNSUPPORTED. EXACT would additionally require that all detection-relevant predicates be represented without relaxation; no such rule occurred in the evaluated Community ruleset.

4.5 Service-Based Rules

The premise $r(x) \Rightarrow H_r(x)$ holds when Snort evaluates the whole rule header, and Snort 3 does not always do so. In Snort 3.12.2.0, before the header of a rule with services is checked, Snort compares the service it has identified for the flow with the rule's services. If the identified service is one of them, the port checks are skipped and the protocol check succeeds unconditionally, while the address checks still run; if a different service has been identified, the rule is rejected; only when no service has been identified is the whole header checked (`src/detection/detection_options.cc` and `src/detection/rtn_checks.cc` in the Snort 3.12.2.0 source). Snort's documentation states a coarser form of this rule: for a service-based rule, evaluation of the header is skipped [17]. Service identification can come from content inspection, so an HTTP request on a port outside `HTTP_PORTS` is still evaluated against the HTTP rules. One form is an

exception. A rule that names a service protocol in its header, such as the twelve rules of the form `alert http ... -> ...` in the evaluated ruleset, is marked as a user-mode rule when Snort builds its header checks, and that restores the port checks (`src/parser/parse_rule.cc` and `src/detection/fp_detect.cc`).

For a rule with a `service` option, the header condition that is necessary for an alert is therefore H_r^{addr}, the address part of H_r, and $r(x) \Rightarrow H_r(x)$ holds only for packets whose service Snort has not identified. The compiler represents these rules with their full header. Section 7.2 measures how many rules this affects and which of the deployed plans still cover the traffic on which they can alert.

4.6 Enforced Packet Domain

The implication above presupposes that the data plane reads the fields the predicate refers to. That presupposition is not free: a parser that assumed a fixed 20-byte IPv4 header would read IPv4 option bytes as transport ports, so a packet satisfying r could fail p_r and the necessary relation would not hold. The implementation therefore makes the precondition explicit rather than assumed.

Let D denote the *supported packet domain*: untagged Ethernet carrying direct IPv4 with version = 4, IHL = 5, fragOffset = 0, flags either zero or `DF`, and no parser error. The P4 parser gates L4 extraction on membership in D, and the ingress control re-checks the same conditions before applying the predicate table. For $x \notin D$, the pipeline executes `packet_domain_fallback`, which marks the packet as a candidate unconditionally and counts it on a dedicated index. Writing $\text{cand}(x)$ for the candidate outcome of the deployed pipeline:

$$r(x) \Rightarrow \begin{cases} p_r(x) \Rightarrow \text{cand}(x), & x \in D, \\ \text{cand}(x), & x \notin D. \end{cases}$$

Consequently, for a rule whose header Snort evaluates in full, no packet satisfying the rule is excluded from Snort inspection, whether or not the parser can interpret it; Section 4.5 identifies the rules for which that premise fails. The data plane fails open by construction: a no-match verdict is emitted only inside D, because no-match is the only outcome that could ever motivate bypassing the IDS. This makes the domain a stated precondition of the selectivity metrics rather than a hidden assumption, and it is why candidate rates in Section 7 are reported over D with domain fallback counted separately.

4.7 Semantic Safety versus Operational Selectivity

A predicate can be semantically safe as a necessary condition while still matching most packets once it is jointly deployed with thousands of other predicates. This distinction is evaluated explicitly in the results.

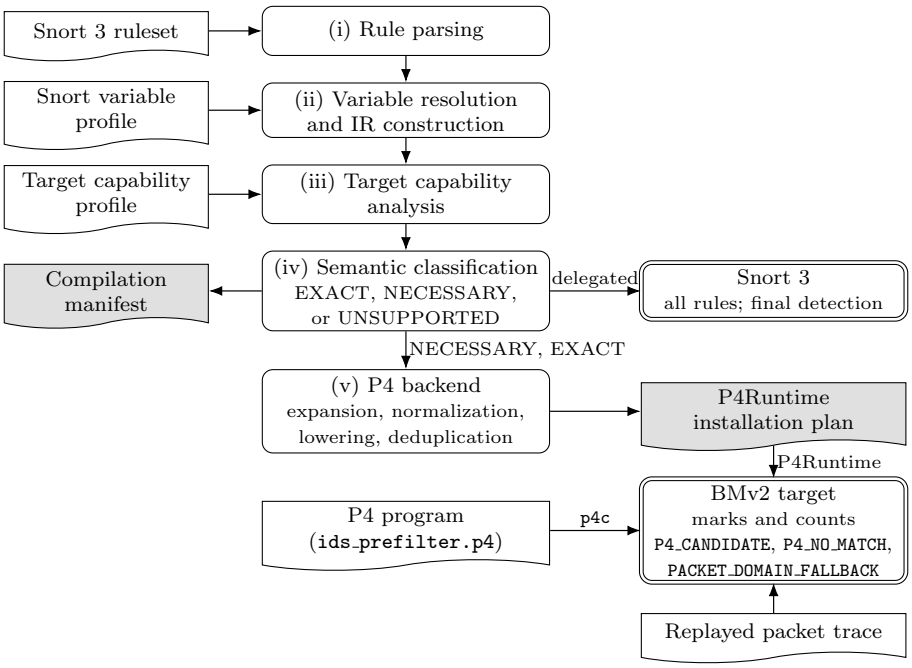


Figure 1: Implemented compilation pipeline and its artifacts. Rounded boxes are the compiler stages (i)–(v); wavy-bottom shapes are inputs (white) and generated artifacts (gray); double-bordered boxes are execution targets. All rules, including UNSUPPORTED ones, remain in Snort 3, which performs final detection. The deployed P4 program marks candidate metadata and counts each packet in one of three categories; it is not integrated into a forwarding or bypass path in front of Snort, and packet traces are replayed into BMv2 for measurement.

5 Proposed Approach

The proposed approach introduces a semantics-aware compilation pipeline between Snort 3 rules and a programmable data plane. Instead of directly converting a complete rule into P4 code, the pipeline first parses and decomposes the rule into an intermediate representation. Each predicate is then analyzed according to the capabilities of the selected P4 target. The result of this analysis determines which components can be represented in the data plane and which must remain delegated to Snort.

The proposed pipeline consists of five main stages: (i) Snort rule parsing, (ii) construction of an intermediate representation, (iii) target capability analysis, (iv) semantic classification, and (v) generation of the corresponding data-plane configuration or IDS fallback. Figure 1 places these stages between their inputs and the BMv2 target.

5.1 Snort Rule Parsing and Intermediate Representation

Stages (i) and (ii) receive Snort 3 rules and extract the elements required for subsequent semantic analysis. Rather than translating the textual rule directly into P4 constructs, the parser converts each rule into a structured intermediate representation (IR). This representation separates the original IDS syntax from the capabilities of a particular data-plane target, allowing the same rule to be analyzed independently of its final implementation.

For the initial prototype, the parser focuses on a restricted rule profile containing the rule action, network protocol, source and destination addresses, source and destination ports, traffic direction, TCP flags, and rule identifier. Additional rule options are also identified when possible, even when they are not executable by the initial P4 backend. This allows unsupported predicates to remain visible during semantic analysis instead of being silently removed from the original rule. Conceptually, the IR of a rule contains the following information:

```
rule:
  sid: 1001
  action: alert
  protocol: tcp
  source:
    address: any
    port: any
  direction: ->
  destination:
    address: 10.0.0.2
    port: 80
  predicates:
    - type: content
      value: "attack"
```

By preserving both supported and unsupported rule components in the IR, the subsequent stages can explicitly determine which semantics are retained by the generated data-plane predicate. The IR therefore acts as an interface between IDS-specific rule parsing and target-specific code generation.

5.2 Target Capability Analysis

Stage (iii) evaluates each predicate represented in the IR against a predefined capability profile of the target data plane. This profile describes which operations and fields are considered executable by the initial P4 backend. Separating target capabilities from the rule parser prevents the frontend from assuming that every parsed IDS construct can be implemented by the programmable switch.

For the initial prototype, the target profile is intentionally restricted to predicates that can be evaluated using packet-header information. The supported subset includes IPv4 addresses and subnets, TCP and UDP ports, IP protocol

information, traffic direction, and TCP flags. Predicates requiring general payload inspection, regular expressions, stream reassembly, arbitrary flow state, or protocol-specific normalization are not considered executable by the initial backend. A predicate marked as unsupported by this profile is not removed

Table 2: Initial capability profile considered by the prototype.

Rule component	**Initial support**
IPv4 protocol	Supported
Source/destination IP	Supported
IP subnet	Supported
TCP/UDP ports	Supported
Traffic direction	Supported
TCP flags	Supported
Payload content	Unsupported
PCRE	Unsupported
Stream reassembly	Unsupported
Arbitrary flow state	Unsupported

from the IR. Instead, its presence influences the semantic classification of the generated representation. This distinction allows the compiler to use supported predicates for pre-filtering while retaining an explicit record of the conditions that must still be evaluated by Snort. Table 3 lists, construct by construct, how the implemented Ethernet/IPv4 profile compiles each form and what it guarantees.

Table 3: Compilation semantics in the implemented Ethernet/IPv4 profile. "Omitted" means absent from p_r while remaining explicit in the manifest and delegated to Snort; it does not mean ignored by the IDS.

Snort construct	**Compiler handling**	**P4 representation**	**Treatment**	**Guarantee / condition**
`tcp`, `udp`, `icmp`, `ip`	Protocol name is assessed; `ip` is a wildcard L4 protocol.	Exact IPv4 protocol ternary key, or no L4 key for `ip`.	Preserved	Within the Ethernet/IPv4 parser scope.
IPv4 address or CIDR; resolved network variable	Resolver substitutes whole-field variables; CIDR is canonicalized to network and prefix length.	IPv4 ternary value/mask.	Preserved	CIDR membership is retained; unsupported or negated expressions do not reach this form.
Exact port or finite port/address list	Resolver flattens lists; finite alternatives are expanded by Cartesian product.	One exact ternary key per concrete alternative.	Preserved / expanded	The union of concrete entries retains the finite-list alternatives, for packets inside the enforced domain D of Section 4.6.
`->` or `<>`	Forward orientation is retained; bidirectional rules swap source/destination addresses, ports, and expansion fields for a reverse entry.	Forward entry; plus reverse entry for `<>`.	Preserved / expanded	Both directions are materialized for `<>`; `<-` falls back.
Supported TCP flags	Flag syntax is normalized to masks; `*` is lowered to a union of ternary entries.	TCP-flag ternary value/mask.	Preserved / expanded	Valid only for forms accepted by the flag normalizer, with TCP protocol validation, and for packets inside D.
Negation, port ranges, unresolved complex header forms	Capability analysis records fallback or unassessed treatment.	Omitted from p_r.	Fallback / delegated	Omission can broaden a predicate; absent another selective H_r, the rule is UNSUPPORTED.
Payload, PCRE, buffers, flow/state, stream, temporal, and application semantics	Ordered options and buffer context remain in the IR and manifest.	No P4 match.	Delegated	Snort remains responsible for these conditions and for final detection.
Service-style or otherwise nonselective rule	No traditional header is invented; a rule without a selective represented header is not emitted.	No P4 entry.	UNSUP.	Entire rule remains in the Snort fallback.
Traditional header with a `service` option	Header is assessed as for a port-based rule; the service is recorded as delegated.	Same keys as the header, ports and protocol included.	Preserved / delegated	Necessary only while Snort evaluates the full header; once the flow's service matches, Snort checks addresses only (Section 4.5).
IPv4 options, fragments, truncated headers, non-IPv4 frames	Outside the compiler's scope; handled by the deployed program rather than by lowering.	No table lookup; `packet_domain_fallback` marks a candidate on a dedicated counter.	Fail-open	Retained for Snort without a no-match verdict, so misparsed L4 fields cannot break $r \Rightarrow$ cand.

5.3 Semantic Classification and Decision

After capability analysis, the compiler determines the semantic class of the generated data-plane representation. The decision considers both the predicates that can be executed by the target and those that remain outside its capability profile. The objective is to prevent a partial translation from being interpreted as an implementation of the complete IDS rule.

If all predicates required by a rule within the considered rule profile can be represented without relaxation, the generated representation is classified as EXACT. If only a subset of necessary predicates can be represented, while additional conditions must still be evaluated by Snort, the representation is classified as NECESSARY. Finally, when no safe and useful predicate can be derived under the target profile, the rule is classified as UNSUPPORTED and remains delegated to the IDS. For example, consider the following simplified Snort rule:

```
alert tcp any any -> 10.0.0.2 80
    (content:"attack"; sid:1001;)
```

The initial target profile can represent the protocol, destination address, and destination port, but does not support the content predicate. Therefore, the generated data-plane predicate corresponds to:

$$p_r = (\text{protocol} = \text{TCP}) \wedge (\text{dstIP} = 10.0.0.2) \wedge (\text{dstPort} = 80).$$

Since matching p_r is not sufficient to establish that the payload contains the string required by the original rule, the generated representation is classified as NECESSARY. Matching packets are therefore candidates for further inspection by Snort rather than detections produced directly by the data plane. The classification result is recorded together with the predicates preserved in the data plane and those delegated to the IDS. This information forms a per-rule compilation manifest, providing an explicit description of the semantic decisions made during translation.

5.4 Compilation Manifest

For each processed rule, the compiler produces a compilation manifest that records the result of the semantic analysis. The manifest identifies the original rule, its assigned semantic class, the predicates preserved by the data-plane representation, and the predicates that remain delegated to Snort. It also records the intended use of the generated component, providing an auditable description of the transformation performed by the compiler.

For the example rule presented previously, a simplified compilation manifest can be represented as follows:

```
rule_id: 1001
classification: NECESSARY
```

```
preserved:
  - protocol: tcp
  - dst_ip: 10.0.0.2
  - dst_port: 80

delegated:
  - content: "attack"

data_plane_action: PREFILTER
final_detection: SNORT
```

The manifest prevents unsupported predicates from being silently discarded during compilation. In addition to supporting debugging and inspection of the generated configuration, it provides information that can later be used to validate whether the behavior observed during execution is consistent with the semantic classification assigned by the compiler.

5.5 P4 Backend, Lowering, and Runtime Installation

The backend lowers target-supported predicates to P4 table matches, deduplicates identical install state, and emits a P4Runtime installation plan. Install-entry provenance retains the associated source-rule IDs for later candidate rule set analysis.

For a finite list, the expansion stage produces the Cartesian product of the approved alternatives; for `<>`, it materializes both source/destination orientations before that expansion. Normalization converts IPv4 CIDRs to network/prefix objects, ports to exact integers, and accepted TCP-flag syntax to masks. TCP-flag lowering retains exact, all-set, all-clear, and at-least-one semantics through ternary matches, with the last represented as a union of entries. P4Runtime encoding then checks field widths and encodes CIDR prefixes as ternary masks.

Deduplication merges only identical normalized or lowered matches, all of which execute the same positive `send_to_ids` action. It therefore does not change the logical union of packets marked candidate, although it does reduce installation state. When identical matches originate from several rules, their `source_rule_ids` are unioned in the retained provenance; otherwise later per-packet candidate-rule-set analysis would lose rule-level information.

The deployed P4 program enforces the packet domain D of Section 4.6 around this table. The parser transitions to L4 extraction only for direct IPv4 with IHL = 5 and no fragmentation, and the ingress control repeats the test, additionally rejecting parser errors, before the table is applied. Three counter indices are kept distinct: `P4_CANDIDATE` for a table hit inside D, `P4_NO_MATCH` for a miss inside D, and `PACKET_DOMAIN_FALLBACK` for traffic outside D, which is marked candidate unconditionally. Collapsing the third category into the second would overstate selectivity, and collapsing it into the first would hide the parser's actual coverage; the metrics below therefore report all three.

5.6 Candidate Rule Provenance and Prototype Scope

The initial prototype deliberately restricts the compilation backend to the capability profile described in Table 2. The objective of this first implementation is not to reproduce the complete Snort 3 rule language in P4, but to evaluate whether explicit semantic classification can support safe rule-derived pre-filtering.

More complex constructs, including general payload inspection, PCRE, stream reassembly, arbitrary stateful dependencies, and protocol-specific normalization, remain outside the initial backend and are delegated to Snort. Future extensions may introduce additional target profiles and backends without requiring changes to the rule representation and semantic-analysis model.

6 Experimental Methodology

6.1 Research Questions and Evaluation Design

The evaluation is designed to investigate two main aspects of the proposed approach: whether the compiler records the intended predicate-level semantic relation between Snort 3 rules and their generated data-plane predicates, and whether the resulting pre-filtering configuration preserves observed alerts within the evaluated scope.

The methodology is divided into three stages. First, a set of Snort 3 rules is processed by the proposed compiler to measure the coverage of the initial target profile and the distribution of semantic classifications. Second, the generated predicates are executed using a P4 software target. Finally, the behavior of the resulting pre-filtering pipeline is compared with the execution of the original rules directly in Snort, both on captured traces and in an end-to-end campaign over synthesized flows (Section 6.5).

6.2 Experimental Environment and P4/BMv2 Target

The initial experimental environment consists of Snort 3 as the reference IDS and a P4 software switch as the programmable data-plane target. The same traffic traces and relevant detection rules are used in both the baseline and the proposed pipeline. The software-based environment is selected to enable reproducible validation of the compilation process before considering hardware-specific optimizations.

The frozen results were produced with Snort 3 version 3.12.2.0, the P4 compiler `p4c` version 1.2.4.2 (build SHA `624c6be80`) invoked through its `p4c-bm2-ss` backend, and BMv2 `simple_switch_grpc` version 1.15.0 (build `68f4a978`), on Ubuntu 22.04 LTS with Python 3.10. The P4Runtime endpoint is `127.0.0.1:9559` with device identifier 0. The data-plane interfaces are `veth` pairs configured with an MTU of 65535, because the replay path writes frames larger than the default MTU, and with IPv6 disabled on each interface, so that router solicitation and multicast traffic generated by the host stack is not

counted by the data-plane counters. The script `scripts/setup_bmv2.sh` in the artifact repository reproduces this configuration.

Two execution paths are compared:

1. **Baseline**: the original traffic is processed directly by Snort using the original rules.

2. **Proposed pipeline**, evaluated by composition rather than as one deployed system: BMv2 classifies traffic with the installed predicates, and Snort processes the same traffic separately with the semantic candidate partition and with the UNSUPPORTED partition. The preservation measures defined below relate the two, by comparing the union of the partition runs with the baseline and by checking the P4 classification of the packets on which candidate alerts fell. No component forwards traffic from BMv2 to Snort.

6.3 Ruleset, Profiles, Traces, and Metrics

The frozen experimental Snort variable profile is documented for reproducibility in Appendix A.

The evaluation considers four main metrics. First, *rule coverage* measures the proportion of analyzed Snort rules for which the compiler can generate an EXACT or NECESSARY data-plane representation. Second, *semantic classification distribution* reports the proportion of rules assigned to each class: EXACT, NECESSARY, and UNSUPPORTED.

Third, *alert preservation* compares the alerts produced by the proposed pipeline with those produced by the baseline Snort execution. For the evaluated traffic, alert preservation is defined as:

$$AP = \frac{|A_{\text{pipeline}} \cap A_{\text{baseline}}|}{|A_{\text{baseline}}|},$$

where A_{baseline} is the set of alerts generated by Snort over the original traffic and A_{pipeline} is the union of the alerts from the two partition runs of the proposed pipeline, the semantic candidate partition and the UNSUPPORTED partition, over the same traffic.

Finally, *candidate traffic* records parser-eligible packets marked candidate or no-match by the P4 prefilter. It is reported as a selectivity measurement, not as measured runtime reduction:

$$CS = \frac{N_{\text{candidate}}}{N_{\text{parser-eligible}}},$$

where $N_{\text{candidate}}$ is the number marked candidate and $N_{\text{parser-eligible}}$ is the parser-eligible population. Here, a parser-eligible packet is one inside the enforced domain D of Section 4.6, that is, one for which the implemented P4 parser produces the Ethernet/IPv4 headers required by the predicate evaluation. A candidate packet matches at least one installed predicate, whereas a no-match packet is parser eligible but matches none. Candidate and no-match rates use

only parser-eligible packets as their denominator. Packets outside D are counted as `PACKET_DOMAIN_FALLBACK`: they are retained as candidates for Snort but are excluded from this denominator, since the prefilter never claims selectivity over traffic it cannot parse.

Packet-domain validation. Because the metric above depends on the parser and the offline matcher agreeing, both are validated directly. A controlled data-plane suite sends one crafted frame per case through BMv2 with the 331-entry plan installed and compares the resulting counter delta with the offline expectation. The eleven cases cover TCP, UDP, and ICMP candidates, a guaranteed no-match, exact and masked TCP flags, IPv4 options (IHL $\neq 5$), a first fragment with MF set, a non-initial fragment, a truncated IPv4 header, and a truncated TCP header. Separately, each full trace is replayed through BMv2 over a raw AF_PACKET socket that transmits the captured Ethernet bytes without reserialization, after a fresh pipeline installation with counters verified at zero and an idle audit confirming a zero delta; sender frame counts are then reconciled against the sum of the three counter categories.

Installation-state reduction. The backend artifact distinguishes `source_entries`, namely the `generated/expanded/normalized` predicate instances before installation-state reduction, from `final_install_entries`, namely the final P4 table entries after deduplication and TCP-flag lowering. Let these quantities be N_{source} and N_{install}, respectively. We measure

$$ISR = \left(1 - \frac{N_{\text{install}}}{N_{\text{source}}}\right) \times 100\%.$$

For the frozen evaluation, the backend reports $N_{\text{source}} = 6675$ and $N_{\text{install}} = 331$, which yields the reported 95.04% reduction. This metric quantifies reduction in P4 installation state only; it does not quantify traffic reduction, Snort rule reduction, execution-time reduction, or throughput improvement.

Provenance-based candidate rule sets.

Backend lowering retains the `source_rule_ids` provenance of each final P4 entry. Let $E(x)$ be the set of retained P4 entries whose predicates match a parser-eligible packet x, and let $S(e)$ be the source Snort rule IDs associated with entry e. The candidate rule set is

$$C(x) = \bigcup_{e \in E(x)} S(e).$$

Thus, multiple source rules may map to one final entry, a packet may match multiple entries, and duplicate IDs are removed by set union. For the frozen strong-discriminator ablation, the P4-eligible baseline is $B = 3611$ rules, and per-packet candidate-rule-space reduction is

$$R(x) = \left(1 - \frac{|C(x)|}{B}\right) \times 100\%.$$

This metric is defined only within the 3,611-rule P4-eligible partition. The 392 operational fallback rules and 14 semantic UNSUPPORTED fallback rules are outside B. A parser-eligible packet with no retained predicate match can have an empty $C(x)$; this is not interpreted as safe full packet bypass because fallback rules remain applicable. Because those 406 rules apply to every packet, we also report the reduction over the full 4,017-rule ruleset,

$$R_{\text{full}}(x) = \left(1 - \frac{|C(x)| + 406}{4017}\right) \times 100\%.$$

Observed preservation measures. We distinguish four finite, empirical checks. *Semantic split preservation* compares FULL Snort alerts with the union from semantic candidate and UNSUPPORTED fallback rules, reporting missing and additional alerts. *P4 alert-packet preservation* checks whether observed parser-eligible packets associated with semantic-candidate alerts are marked P4_CANDIDATE, reporting candidate alert packets, parser eligibility, P4 candidates, and P4 no-matches. *Strong-discriminator observed-event coverage* checks retained-predicate coverage for P4-eligible alert events while accounting for operational and semantic fallback events. Finally, *candidate-set SID membership* checks whether an observed P4-eligible alert SID belongs to $C(x)$. The last measure does not validate dynamic stateful Snort execution using $C(x)$, because prior packets, stream state, flowbits, and other stateful semantics may be required.

6.4 Validation Procedure and Held-Out Protocol

The evaluation includes a controlled validation and CICIDS2017 [13] Thursday web-attack windows. SQLi and XSS are development/diagnostic windows; Brute Force is reserved as held-out evaluation. Frozen policy and grouping configurations are evaluated on Brute Force without retuning after its inspection. The initial scenarios considered controlled cases designed to exercise the semantic classes produced by the compiler. Rather than attempting to evaluate the complete Snort 3 rule language, the experiments focus on representative rules that expose different relationships between the original IDS rule and the generated data-plane predicate.

The first scenario is a legacy semantic-model unit scenario for a header-only EXACT-class expectation. This is a component-level test specification, not a trace-based EXACT result for the frozen Community ruleset. That ruleset produced EXACT= 0; the reported experiments therefore do not claim empirical EXACT validation.

The second scenario considers rules that combine supported header predicates with unsupported conditions, such as payload content. These rules are expected to produce NECESSARY representations. The generated data-plane predicates

are used only for pre-filtering, while Snort retains responsibility for evaluating the remaining conditions and producing the final detection decision.

The third scenario considers rules for which the initial target profile cannot derive a useful and safe pre-filtering predicate. Such rules are classified as UNSUPPORTED and remain entirely delegated to Snort. This scenario verifies that unsupported semantics are explicitly reported rather than silently removed during compilation.

6.5 End-to-End Differential Campaign

The trace-based checks exercise only the few rules that fire in the evaluated windows. To test the safety relation on a wider set of rules, a differential campaign uses Snort 3 as the oracle and BMv2 as the system under test. For each rule, a generator synthesizes from the rule's IR a short flow whose witness packet should make Snort raise that rule's alert: a TCP handshake when the `flow` option requires an established session, then a witness that carries the rule's header values, places each `content` in the inspected buffer, and uses a service port that lets Snort bind the required inspector. Each witness has two neighbors: a payload-negative frame with the same 5-tuple but without the content, on which Snort is expected not to alert, and, where the installed entries leave one, a header-negative frame whose destination port falls outside every installed entry.

A witness could be synthesized for 2,142 of the 4,017 rules. The other 1,875, including the 14 UNSUPPORTED rules, were excluded: 1,654 of them carry options such as `pcre`, `byte_test`, `byte_jump`, or `flowbits` that cannot be satisfied by choosing bytes, and the rest have content the generator cannot place or a header it cannot resolve. A sample of 1,000 rules, stratified proportionally by protocol, service, and direction with a fixed seed, was written to a trace in which each case occupies its own second: 6,684 frames, all inside D. The same sample was regenerated whenever the witness generator was revised.

Snort 3 reads the trace with the full ruleset and the experimental profile, and each alert is attributed to a case twice, by packet range and by timestamp. BMv2 receives the same frames one at a time, and the counter category that increments after each frame is recorded. Each case then receives one verdict. It is CONFIRMED when Snort alerted and both the witness and every frame it alerted on were marked candidate. It is a VIOLATION when Snort alerted but the witness, or another frame that satisfies the rule's header and on which Snort alerted, was marked no-match; this is the only outcome that contradicts $r \Rightarrow \text{cand}$. It is VACUOUS when Snort did not alert, so the case did not test the implication. It shows STATE DEPENDENCE when the witness was a candidate but Snort attributed the alert to a session frame that does not satisfy the rule's header and that the data plane marked no-match. Rules placed in operational fallback by the strong-discriminator policy are reported separately, because the prefilter is not responsible for them.

The campaign was run against both the 331-entry global plan and the 322-entry strong-discriminator plan. The global plan limits what the campaign can falsify: entry `p4-106` matches every in-domain TCP packet, so a TCP witness

is a candidate however its own rule was lowered. To quantify this, the offline matcher also records whether each witness is matched only by entries whose provenance contains the case's own rule. For such a case, a lowering defect in that rule would have produced a VIOLATION.

7 Results

7.1 Semantic Compilation Coverage (RQ1)

The frozen compilation processed and successfully parsed all 4,017 rules in the evaluated Snort 3 Community ruleset. The semantic manifest contains no EXACT classifications, 4,003 NECESSARY classifications, and 14 UNSUPPORTED classifications. Hence, 99.65% of the rules contributed a candidate predicate under the supported profile. For the 1,060 port-based NECESSARY rules, and for the 12 that name a service protocol in their header and keep their port checks, that predicate is a necessary condition of the rule. For the 2,928 with a `service` option it is necessary only on traffic whose service Snort has not identified (Section 7.2). The remaining three use service-specific options without a `service` option, and their status was not confirmed.

This percentage characterizes semantic candidate coverage, not complete offloading of Snort semantics. For a rule r and its NECESSARY predicate p_r, the relation the compiler targets is $r \Rightarrow p_r$; neither $p_r \Rightarrow r$ nor $r \Leftrightarrow p_r$ is assumed. Payload inspection, stream and flow/state semantics, normalization, and other target-inexpressible conditions remain delegated to Snort. For the 14 UNSUPPORTED rules, no safe selective P4 predicate was established, and the rules remain in the Snort fallback rather than being ignored. The absence of EXACT rules therefore reflects a deliberate conservative classification policy: the compiler does not declare equivalence when it has preserved only a necessary condition.

The frozen manifest explains the 14 UNSUPPORTED cases without silently discarding them. Four are traditional rules whose represented headers are nonselective while payload or byte-level conditions remain delegated; the other ten are service-style rules whose service, flow, buffer, and payload semantics are delegated and for which the parser intentionally invents no traditional packet headers. Thus, each lacks a selective represented H_r and remains in the Snort fallback. Across NECESSARY rules, the manifest commonly records content and other payload/buffer conditions, flow or state conditions, application/service semantics, and PCRE as delegated while retaining at least one selective represented header predicate. A smaller set also records a fallback header form, such as a port range, negated port, or unsupported flag encoding, but is emitted only when another selective represented predicate is available.

Answer to RQ1. Under the evaluated target profile and within the enforced packet domain, the compiler derives explicit necessary predicates for the 1,072 NECESSARY rules whose header ports Snort checks, while making the remaining semantic limitations visible; traffic outside that domain is retained by failing

open rather than by an unstated parsing assumption. For the 2,928 rules with a `service` option, the derived predicates are necessary only where Snort evaluates the full header; a predicate restricted to the header addresses would be necessary for all of them and still selective for 2,921.

7.2 Service-Based Rules and Plan Coverage

Of the 4,003 NECESSARY rules, 2,928 declare a `service` option, most often `http` (2,314), `imap` (150), `pop3` (138), and `smtp` (131). Every one of their predicates constrains the protocol, and 2,877 also constrain a port. By Section 4.5, Snort can raise such a rule on a packet whose ports or protocol differ from the rule header once the flow's service is identified, so the rule's own predicate can be false on a packet it alerts on. Fifteen further rules are service-based without a `service` option and are not counted: twelve name `http` in their header and keep their port checks, and three use service-specific options through a mechanism that was not confirmed in the source.

Whether this becomes a missed candidate depends on the whole plan. For each of the 2,928 rules, the region on which Snort can alert once the service is identified is the rule's address condition with any port and any protocol. In the 331-entry global plan, the port- and protocol-free entries `p4-40` and `p4-138` contain that region for 2,919 rules, and `p4-106` contains its TCP part for the remaining nine: SIDs 1415 and 1416 (SNMP over UDP to the broadcast address), 1941 (TFTP over UDP), 28852, 29174, and 38993 (HTTP), and 56583 to 56585 (Kerberos), all with unrestricted addresses. The global plan is therefore candidate-complete for these rules except for non-TCP packets of those nine services that no entry of the rule itself matches. The 322-entry strong-discriminator plan removed the nine entries that constrain at most the protocol and exactly `HOME_NET`; the six port-free entries it keeps carry TCP flags or other addresses, and none contains the region of any of the 2,928 rules. Of those rules, 2,877 are among its 3,611 P4-eligible rules.

Compiling the rules as Snort 3 evaluates them changes the structural results substantially. The compiler provides this as an option: ports and protocol of service-based rules are delegated, and the protocol is kept only for rules whose `flags` option already restricts them to TCP. Seven rules lose their last selective predicate, giving 3,996 NECESSARY and 21 UNSUPPORTED rules, and the backend produces 235 final entries instead of 331. Applied unchanged, the strong-discriminator policy keeps only 739 rules P4-eligible and places 3,257 in operational fallback, because an address-only predicate on `HOME_NET` is not a strong discriminator; applied to the frozen artifacts, the same computation reproduces the frozen 3,611, 392, and 322. Under the accounting of Table 6, in which fallback rules apply to every packet, no packet's candidate set could then fall below 3,278 rules, which bounds the full-ruleset reduction at 18.40%, against the 81.81% measured on held-out Brute Force with the frozen predicates. These are structural results computed without traffic; the trace-level rates under the corrected predicates were not measured.

No experiment in this paper was designed to exercise this case. Under the

global plan, an alert of this kind would fall on a packet that `p4-40`, `p4-138`, or `p4-106` already marks candidate, so the trace-based checks could not reveal it; only the offline strong-discriminator audit of the held-out trace could, and it found no missing event. The end-to-end campaign places each witness on a port of the rule's own header. The measurements that follow are therefore correct for the traffic they observed, but the strong-discriminator ablation and the per-packet candidate rule sets derive from predicates that are not necessary conditions for service-based rules.

7.3 P4 Backend Feasibility (RQ2)

Table 4 summarizes the semantic compilation and backend feasibility results. The backend generated, expanded, and normalized 6,675 predicate instances. These instances yielded 327 unique matches before TCP-flag lowering. Two flag-constrained groups required expansion, adding four entries and resulting in 331 final installation entries. The P4Runtime plan encoded 331/331 entries, with zero encoding failures and zero validation errors, within the 8,192-entry table capacity. Installation and read-back validation of the 331-entry plan was reported for the evaluated BMv2 target.

The reduction from 6,675 source instances to 331 final entries (95.04%) is an *installation-state reduction*: identical backend matches are merged before installation. It does not represent traffic reduction, elimination of Snort rules, execution-time improvement, or speedup.

Answer to RQ2. The semantically derived predicates were lowered to the evaluated target within its table capacity and P4Runtime validation constraints, without concealing unsupported semantics.

Table 4: Semantic compilation and backend feasibility for the frozen profile. The 95.04% value is backend installation-state reduction, not traffic reduction.

Rules	Exact	Necessary	Unsupported	Predicate instances	Unique	Final	P4Runtime
4017	0	4003	14	6675	327	331	331/331

7.4 Packet-Domain Enforcement and Validation

Because the selectivity and candidate-set metrics are defined over the enforced domain D, the correspondence between the deployed parser and the offline matcher was validated before those metrics are reported.

The controlled data-plane suite passed all eleven cases with exact counter deltas. TCP, UDP, and ICMP candidates, a guaranteed no-match, and exact and masked TCP flags were classified inside D as expected. IPv4 options, a first fragment with MF set, a non-initial fragment, a truncated IPv4 header, and a truncated TCP header were each routed to `PACKET_DOMAIN_FALLBACK`, with

no candidate or no-match count. No case produced a data-plane verdict that disagreed with the offline expectation.

Across the evaluated traces, 350 of 35,274 SQLi frames, 2,772 of 191,848 XSS frames, and 7,199 of 1,108,658 Brute Force frames fell outside D. These are dominated by non-IPv4 EtherTypes: ARP (125, 1,660, and 4,074), IPv6 (195, 820, and 2,523), and smaller LLDP and non-IP EtherType populations. Only the held-out trace contains direct-IPv4 frames excluded by the header gate itself: 16 frames with fragmentation or reserved flags. They are the reason its parser-eligible population is 1,101,459 rather than the 1,101,475 obtained when fragmentation is not gated; all 16 remain candidates through domain fallback.

Trace replay accounting was reconciled against the data plane. SQLi and XSS agree frame for frame: 35,274 and 191,848 frames sent, with identical totals across the three counters and exact per-category agreement with the offline classifier. The Brute Force replay recorded 1,108,571 of 1,108,658 sent frames, a 0.008% shortfall confined to the candidate category, with domain fallback exact at 7,199 and no-match at zero. That residue is a transport-layer measurement loss on a 23-minute replay rather than a classification disagreement, so the offline classifier, which covers the complete trace population, is used as the primary source for the held-out quantities.

Revalidating the frozen analyses under the enforced domain reproduced the reported results. On the held-out trace the global candidate rate remains 100% of D, the strong-discriminator rate 98.942%, the mean candidate set 324.52 rules, and the mean candidate-rule-space reduction 91.01%; in all three traces every candidate-alert packet is still a candidate. No policy or grouping configuration was retuned during this revalidation.

7.5 Global Binary Prefilter Selectivity (RQ3)

Table 5 reports packet-level selectivity under the current Ethernet/IPv4 parser scope and supported protocols. The original global candidate bit marked 100% of parser-eligible packets as candidates in SQLi, XSS, and the held-out Brute Force trace. Parser-ineligible frames are outside this denominator and are not treated as P4 no-matches. Thus, within the evaluated CICIDS2017 windows, high semantic candidate coverage did not translate into useful packet-level bypass selectivity for the global binary candidate bit.

7.6 Selectivity Causal Analysis

The causal audit first identified broad NECESSARY predicates in the global table. Entry `p4-106` matches TCP protocol only; it subsumes 246 installation entries (74.32%). Entry `p4-40` matches traffic with destination `192.168.10.0/24` and subsumes 210 entries (63.44%), whereas `p4-138` matches the corresponding source prefix and subsumes 107 entries (32.33%). These predicates individually cover large fractions of the evaluated traffic. Deduplication is not responsible for the coverage, because it merges identical normalized matches rather than

Table 5: Packet-level selectivity. Candidate rates are among parser-eligible packets, that is, packets inside the enforced domain D; frames outside D are retained as candidates through domain fallback and are excluded from the denominator. The strong-discriminator columns are a frozen operational ablation computed with the offline classifier, not replayed on BMv2.

Trace	Total	Eligible	Global candidate	Global no-match	Strong candidate	Strong no-match
SQLi	35274	34924	34924	0	99.015%	0.985%
XSS	191848	189076	189076	0	98.869%	1.131%
Brute Force (held-out)	1108658	1101459	1101459	0	98.942%	1.058%

removing restrictions; TCP-flag lowering is also not responsible, because these broad entries do not depend on TCP flags.

The frozen strong-discriminator ablation treats an installation entry as a strong discriminator when it constrains a source or destination port, the TCP flags, or an IPv4 prefix other than exactly `HOME_NET`; a rule remains P4-eligible only if every final entry derived from it is strong. The criterion is structural and uses no traffic frequencies. It removed nine entries, each constraining at most the protocol and exactly `HOME_NET` (`p4-3`, `p4-5`, `p4-20`, `p4-24`, `p4-40`, `p4-80`, `p4-106`, `p4-138`, and `p4-286`), retaining 322 installation entries for 3,611 P4-eligible rules. It places 392 semantic candidates in operational fallback and retains the 14 semantic UNSUPPORTED rules in fallback; it is therefore an operational ablation, not a revised semantic classification. Its trace-level rates were computed with the offline classifier; the 322-entry plan was installed on BMv2 only for the end-to-end campaign of Section 7.10. As shown in Table 5, the candidate rate nevertheless remained between 98.869% and 99.015% on the development traces and 98.942% on the held-out trace, where 11,654 parser-eligible packets were no-matches.

The residual-selectivity audit shows that removing obviously broad predicates does not eliminate distributed union coverage from the retained predicates. Accordingly, semantic safety and operational selectivity are distinct properties: a predicate can be sound as a necessary condition while adding little packet-level selectivity when composed with thousands of other necessary predicates.

Answer to RQ3. The global binary prefilter is conservative, except for non-TCP traffic of nine service-based rules, but operationally non-selective on the evaluated traces. The frozen broad-predicate ablation remains nearly as non-selective and, by removing the port-free entries, is not conservative for service-based rules (Section 7.2).

7.7 Candidate Rule-Set Reduction (RQ4)

The global candidate bit asks whether a packet remains a candidate for IDS handling. The provenance-based analysis asks a different question: which of the 3,611 P4-eligible Snort rules remain plausible for a given packet? For each

Table 6: Offline per-packet candidate rule set feasibility using the frozen strong-discriminator baseline of 3,611 P4-eligible rules. The last column counts the 406 fallback rules, which apply to every packet, against the full 4,017-rule ruleset.

Trace	Base.	Mean	Median	P90	P95	P99	Max	Mean reduction	Full-ruleset reduction
SQLi	3611	179.28	27	799	799	799	1228	95.04%	85.43%
XSS	3611	140.49	13	799	799	1227	1228	96.11%	86.40%
Brute Force (held-out)	3611	324.52	241	799	799	1227	1228	91.01%	81.81%

parser-eligible packet, it unions the source-rule IDs of the matched retained P4 entries to derive a per-packet candidate rule set.

The mean candidate-rule-space reduction was 95.04% for SQLi and 96.11% for XSS; the SQLi value coincides with, but is unrelated to, the 95.04% installation-state reduction in Table 4. On held-out Brute Force, the mean set contained 324.52 rules, a mean reduction of 91.01%, so the effect carried over to the held-out trace with a smaller magnitude. Brute Force was not used for policy retuning. Counting the 406 fallback rules that apply to every packet, the mean reduction over the full ruleset is 85.43% for SQLi, 86.40% for XSS, and 81.81% for held-out Brute Force.

This is an offline candidate-rule-space feasibility measurement, not a measured reduction in Snort execution, throughput, or latency, and not a count of rules dynamically executed by Snort. A candidate-set-aware Snort execution engine was not implemented. The sets are also derived from the port-constrained predicates of Section 7.2: a packet of a service carried on a port outside a service-based rule's header can omit that rule from $C(x)$ although Snort would evaluate it. Deriving the sets from address-only predicates for those rules would make them larger, so the reported reductions are an upper bound on that derivation; under the strong-discriminator policy it could save at most 18.40% of the full ruleset (Section 7.2).

7.8 Coarse-Grained Grouping Ablation (RQ4)

The frozen grouping study assigned the 322 retained entries to groups by protocol (4 groups), protocol plus direction (12), protocol plus service (268), and protocol plus direction plus service (308). Its per-packet candidate rule set, however, is computed at entry level, as the union of the provenance of the matched entries, and does not read the group assignment. The four strategies therefore report the values of Table 6 by construction; that they coincide is not an experimental observation, and the study does not measure the candidate set that a grouped prefilter would produce.

What the study does record is the structure of the groups, and that structure bounds the cost of grouping. Under protocol grouping, the four groups partition the 3,611 rules without overlap, and the TCP group alone holds 3,439 of them. A grouped prefilter that forwarded every rule of an activated group would give each packet that activates the TCP group at least 3,439 candidate rules, a reduction

of at most 4.8%, where entry-level provenance gives 91–96% on average. Finer strategies shrink the largest groups but do not remove the effect: under protocol plus service, the TCP group for destination port 80 holds 2,024 rules, and 2,358 rules belong to more than one group, up to nine. A per-packet grouped candidate set over the traces was not computed.

The 91–96% reductions in Table 6 therefore come from the mapping of matched P4 entries to their source-rule IDs, followed by the union of those provenance sets for each packet, and not from grouping.

7.9 Dynamic Alert Preservation (RQ4)

Table 7 separates four empirical properties. They are semantic split preservation, P4 alert-packet preservation, observed-event coverage under the strong discriminator, and entry-level candidate-set SID membership. These are not interchangeable claims of equivalence.

For SQLi, XSS, and held-out Brute Force, the semantic split had zero missing or additional alerts, and all observed parser-eligible candidate-alert packets matched the P4 table. The P4 columns of Table 7 were measured on BMv2 by replaying only the candidate-alert packets of each trace, before the packet-domain gate of Section 4.6 was added to the data plane. Re-classifying the same packets offline under the enforced domain gives the same counts, with none falling into domain fallback; the data-plane replay was not repeated under the corrected program. No UNSUPPORTED rule fired in any of the three traces, which limits the discriminating power of the semantic split comparison; Section 9 states this explicitly. Thus, observed alert preservation was 100% in the three traces. In held-out Brute Force, FULL Snort produced 2,391 alerts from five SIDs (254, 399, 402, 650, and 1394). All 2,391 candidate-alert packets were parser eligible and P4 candidates; none was a P4 no-match. The offline strong-discriminator audit covered 2,289 eligible alert events through retained predicates and 102 through fallback, with zero missing. Observed candidate-set SID membership was 2,289/2,289.

The trace names do not imply that the observed Snort alerts detected the corresponding CICIDS2017 attack labels. These measurements describe the evaluated ruleset behavior and constitute observed alert preservation, not formal equivalence, guaranteed preservation for arbitrary traffic, or detection accuracy.

Table 7: Observed alert preservation. Missing and additional compare the semantic split with FULL Snort; P4 columns concern candidate-alert packets, replayed alone on BMv2 before the packet-domain correction and reproduced offline under the enforced domain.

Trace	FULL	Cand.	Unsup.	Missing	Add.	Alert packets	P4 cand.	P4 no-match	Pres.
SQLi	150	150	0	0	0	150	150	0	100%
XSS	528	528	0	0	0	528	528	0	100%
Brute Force (held-out)	2391	2391	0	0	0	2391	2391	0	100%

7.10 End-to-End Differential Campaign (RQ4)

Table 8 reports the campaign verdicts. No case produced a VIOLATION under either plan, and no frame fell outside D, so no case was resolved by domain fallback. The two attributions of Snort alerts to cases, by packet range and by timestamp, agreed for every case.

Under the global plan, 923 of the 1,000 rules were CONFIRMED and 77 were VACUOUS. All 1,000 witnesses and all 1,000 payload-negative frames were candidates, and all 101 header-negative frames were no-matches. Every confirmed witness was matched by its own rule's entries but also by an entry of another rule, so no confirmation under this plan can be attributed solely to the rule's own predicate. This is the broad coverage of Section 7.6 seen at the level of individual rules: the same coverage that removes packet-level selectivity also removes the power to falsify a single rule's lowering.

Under the strong-discriminator plan, 851 rules were CONFIRMED, 107 were in operational fallback, 40 were VACUOUS, and 2 showed STATE DEPENDENCE. Every confirmed witness was matched by at least one entry of its own rule, and for 153 of the 851 it was matched by no other entry, so a lowering defect in those rules would have been observed as a VIOLATION; the other 698 were matched by their own entries together with entries of other rules. Of the 107 fallback rules, 66 had witnesses marked no-match, as expected for rules with no retained entry, and 41 were candidates through other rules' entries. In both runs, the category predicted by the offline matcher agreed with the BMv2 counter on all 2,101 witness and neighbor frames, which extends the eleven-case packet-domain suite of Section 7.4 to a thousand rules and to the matcher that produced the selectivity results.

Under the strong-discriminator plan, the payload-negative frames measure the conservative excess of a header-level prefilter: 927 of them were candidates on which Snort did not alert. Seven more were candidates on which Snort did alert, because their rules carry no `content` to remove, and the remaining 66 belong to the fallback rules, which the plan does not prefilter. The two STATE DEPENDENCE cases, SIDs 43809 and 47386, are HTTP request rules. Snort attributed each alert to the server-to-client acknowledgment that triggers inspection of the buffered request, and the strong-discriminator plan marks that acknowledgment no-match because it does not satisfy the rule's header. The compiled predicate held on the witness, but a deployment that withheld no-match packets from Snort could delay or suppress these alerts. The global plan hides this dependence, because `p4-138` matches every packet leaving `HOME_NET`.

Answer to RQ4. The evaluated predicates preserved all observed candidate alert events in the three traces. In the end-to-end campaign no case was a VIOLATION: no rule that a plan prefilters had its witness marked no-match while Snort alerted, across 923 confirmed rules under the global plan and 851 under the strong-discriminator plan. Under the strong plan, Snort also alerted on no-match frames for the 66 fallback rules it does not prefilter and for the two state-dependence cases. The predicates also exposed substantially smaller per-packet candidate rule sets; that result remains an offline feasibility measurement

Table 8: End-to-end differential campaign over one stratified sample of 1,000 rules. *Own-only* counts confirmed rules whose witness was matched only by entries of their own rule, computed with the offline matcher. The global row used an earlier revision of the witness generator, and an IR without two later `content` repairs, than the strong-discriminator row; those revisions changed witness construction only, reduced vacuous cases, and none produced a violation.

Plan	Entries	Confirmed	Violation	State dep.	Oper. fallback	Vacuous	Own-only
Global	331	923	0	0	–	77	0
Strong discriminator	322	851	0	2	107	40	153

because dynamic stateful candidate-set execution was not implemented, and it rests on port-constrained predicates for service-based rules. With Snort 3's service semantics, the strong-discriminator policy keeps 739 eligible rules, which bounds the full-ruleset reduction at 18.40% (Section 7.2).

Figure 2 summarizes the semantic compilation flow and distinguishes the implemented binary prefilter from the offline provenance analysis.

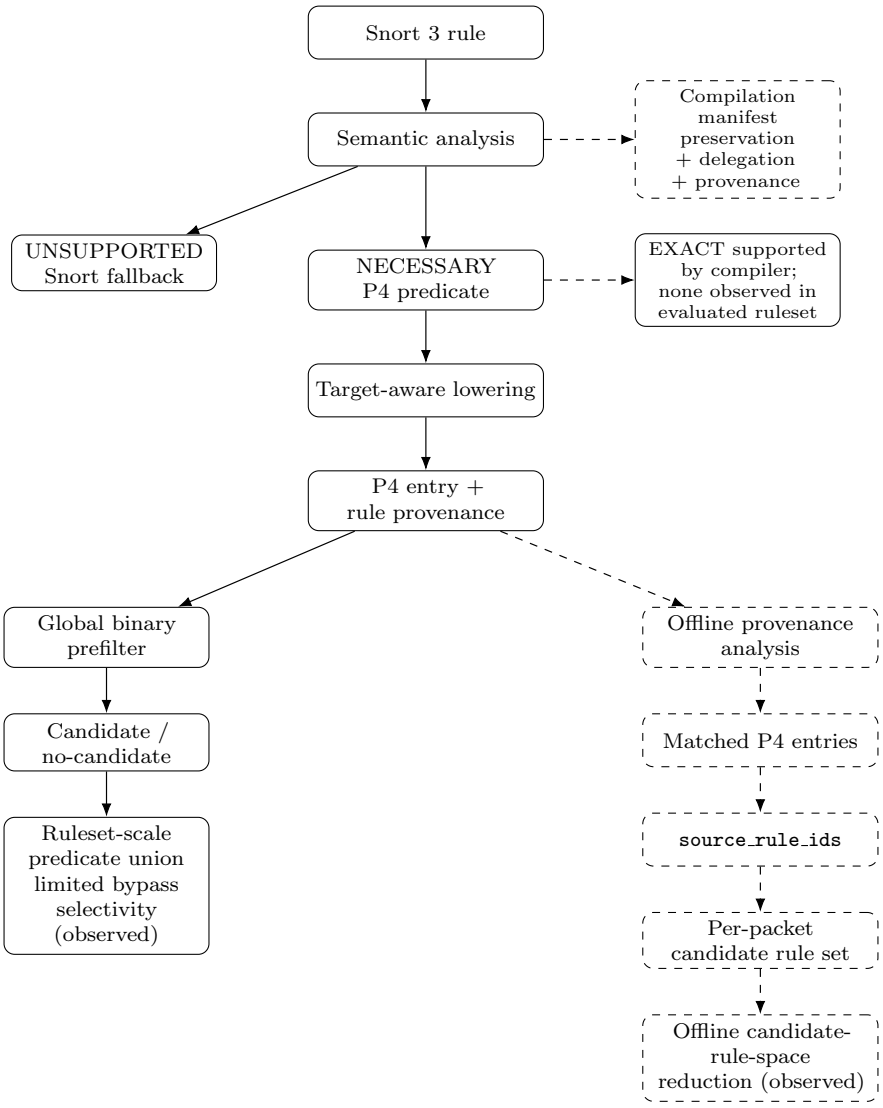


Figure 2: Semantics-aware, target-aware compilation overview. Semantic analysis retains unsupported or delegated semantics in Snort while deriving necessary P4 predicates; for rules with a `service` option they are necessary only where Snort evaluates the full header (Section 4.5). Target-aware lowering preserves source-rule provenance. The implemented global binary prefilter aggregates predicates into candidate metadata and showed limited packet-bypass selectivity in the evaluated ruleset. The dashed provenance branch is an offline feasibility analysis for deriving per-packet candidate rule sets; dynamic candidate-set-aware IDS execution is not implemented. EXACT is supported by the classifier but was not observed in the evaluated ruleset.

8 Discussion

8.1 Semantic Safety Does Not Imply Operational Selectivity

For a Snort rule r and a generated P4 predicate p_r, the relation $r \Rightarrow p_r$ states that p_r is a safe necessary condition under the supported semantic profile and inside the enforced packet domain D, with traffic outside D retained through fail-open fallback. It does not establish $p_r \Rightarrow r$ or $r \Leftrightarrow p_r$. Semantic safety therefore asks whether traffic that satisfies the rule also satisfies its compiled predicate; operational selectivity asks how much other traffic that predicate can exclude.

Under the evaluated profile, 4,003 of 4,017 rules were classified as NECESSARY, 14 as UNSUPPORTED, and none as EXACT. Nevertheless, the original global binary prefilter marked 100% of parser-eligible packets as candidates in SQLi, XSS, and held-out Brute Force. This is not a contradiction. The usefulness of a safely compiled predicate is not determined solely by whether it preserves a necessary condition, but also by how that predicate composes with predicates generated for the remainder of the ruleset. The absence of EXACT classifications is both a limitation of the evaluated target/profile and evidence that partial semantic preservation was not promoted to equivalence.

8.2 The Union Effect in Large Rule Sets

Conceptually, the global candidate decision is the disjunction $P_{\text{global}}(x) = p_1(x) \vee p_2(x) \vee \cdots \vee p_n(x)$. Individual predicates may be useful filters, but the aggregate coverage of a large disjunction can approach the workload population. The causal audit identified broad initial contributors: `p4-106` matches TCP only, `p4-40` matches a destination in `HOME_NET`, and `p4-138` matches a source in `HOME_NET`.

Broad predicates were an initial contributor, but they were not a complete explanation. The frozen strong-discriminator ablation retained 322 entries for 3,611 P4-eligible rules, placed 392 rules in operational fallback, and retained 14 semantic UNSUPPORTED rules in fallback. Even then, the held-out Brute Force trace had a 98.942% candidate rate among parser-eligible packets. The residual audit indicates that near-universal coverage persisted through the distributed union of many retained predicates. Thus, the evidence does not attribute poor selectivity to only three broad entries. It instead indicates that large heterogeneous rulesets can make a single global candidate bit too coarse and discard rule-level provenance.

The union also limits what can be tested. In the end-to-end campaign, every confirmed witness under the global plan was also matched by another rule's entry, so a defect in any single rule's lowering would have been masked by the rest of the table. Only after the broad entries were removed did 153 confirmations depend on the rule's own entries alone. Testing the lowering of an individual rule therefore needs a configuration in which that rule's predicate is not covered

by the others, a condition closely related to the one that packet-level selectivity depends on.

8.3 From Packet Bypass to Candidate-Rule Selection

The global prefilter asks whether a packet should remain a candidate for IDS handling. Provenance enables a different question: which IDS rules remain plausible candidates for that packet? The latter is obtained by mapping matched P4 entries to their recorded source-rule IDs and taking their union as a per-packet candidate rule set. Rule-level provenance consequently retains information that a global candidate bit discards.

Using the frozen 3,611-rule strong-discriminator baseline, the mean candidate-rule-space reduction was 95.04% for SQLi and 96.11% for XSS. For held-out Brute Force, the mean candidate set contained 324.52 rules (median 241; P95 799; P99 1,227; maximum 1,228), corresponding to a mean reduction of 91.01%. The held-out trace suggests that this candidate-set effect was not restricted to the development traces, although its magnitude was traffic-dependent and lower than the approximately 95–96% observed on SQLi and XSS. The 406 fallback rules bound what any candidate set can save: counted against the full ruleset, the reductions are 85.43%, 86.40%, and 81.81%, and no packet can fall below 406 applicable rules.

The service-based case narrows this further. Once ports and protocol of service-based rules are treated as Snort 3 treats them, most of those rules keep only an address condition on `HOME_NET`, and the strong-discriminator policy leaves 739 of the 3,996 NECESSARY rules P4-eligible, so a header-derived candidate set could save at most 18.40% of the full ruleset for any packet. Much of what header provenance appeared to discriminate was the header port, which Snort 3 does not use once it knows the service. A candidate selector that also used the identified service, which Snort itself uses to reject rules of other services, is a natural next step; this paper does not evaluate one.

This observation motivates a future design space involving rule masks, rule partitions, hierarchical selectors, candidate metadata, and dependency-aware scheduling in hybrid P4/IDS systems. It is not evidence of a runtime implementation: candidate-set-aware Snort execution was not implemented, no throughput or latency benchmark was conducted, and no speedup is demonstrated. In particular, dynamic stateful rule execution remains unresolved.

8.4 Why Coarse Grouping Is Unlikely to Help

The 91–96% candidate-rule-space reduction was produced by fine-grained entry-to-source-rule provenance, not by grouping. The frozen grouping study does not show this by comparison, because its candidate-set metric ignores the groups and is identical across strategies by construction. The group structure it records points the same way with more force: a coarse group forwards every rule it contains, and under protocol grouping the TCP group holds 3,439 of the 3,611

rules, so coarse grouping would discard most of the discrimination that entry-level provenance retains rather than add to it.

Coarse structural groups aggregate rules whose necessary predicates remain different. The service-oriented groupings also overlap heavily, with 2,358 rules in more than one group. This does not show that every grouping is unhelpful, and the grouped candidate set was not measured on the traces. Future grouping designs could account for provenance, semantic dependencies, rule overlap, flow/state dependencies, and hardware resource constraints.

8.5 Implications for P4-Based IDS Offloading

The evaluated results suggest that translation rate alone is insufficient for assessing a P4-based IDS prefilter. Classifying 99.65% of rules as NECESSARY did not imply operationally selective binary packet bypass. This complements prior work on rule-derived P4 filtering and resource-aware data-plane representations [7, 14]: the semantic relation between the original rule and the compiled predicate must remain explicit.

Semantic transparency should therefore be a first-class compiler output. A useful artifact records preserved predicates, delegated predicates, fallback rules, and unsupported rules rather than presenting partial translation as full rule offload. The evidence also supports evaluation along multiple axes: semantic coverage, target feasibility, installation state, packet-level selectivity, observed alert preservation, and candidate-rule-space reduction. Work on context required by rule-based prefiltering further reinforces the need to account for IDS state beyond packet-header matching [3].

The service-based case shows that even the header of a rule is not self-describing. What a header condition means depends on how the IDS evaluates it, and Snort 3 drops the port and protocol conditions of rules with a `service` option once a flow's service is known. A prefilter that derives port conditions from Snort 3 rule headers can therefore exclude packets the rule would alert on, unless its plan keeps port-free coverage for the addresses those rules name. In the evaluated global plan, the broad entries that removed packet-level selectivity are also what kept the plan candidate-complete for all but nine of these rules: safety and selectivity pulled against each other within the same entries.

Binary packet bypass may be too coarse for large heterogeneous rulesets under the evaluated configuration. Predicate provenance can instead be treated as a compiler output in its own right: it can inform downstream rule-selection mechanisms even where a binary candidate decision is not selective. This is an architectural implication, not an implemented acceleration or a claim about current Snort capabilities. It is consistent with the broader interest in coordinated programmable-data-plane and IDS processing [5].

9 Threats to Validity and Limitations

9.1 Construct Validity

The reported metrics characterize different constructs and should not be collapsed into a single notion of offloading effectiveness. Semantic coverage measures whether a safe P4 predicate was derived under the supported profile; it does not measure semantic equivalence. In particular, the 4,003 NECESSARY classifications do not constitute full Snort rule offload, and EXACT=0 means that no evaluated rule had all of its semantics represented by the P4 predicate. This is both a limitation of the evaluated target/profile and a consequence of the conservative classifier, which does not label partial preservation as equivalence.

The NECESSARY classification itself rests on a model of Snort's header evaluation that holds for port-based rules but not for rules with a `service` option (Section 4.5). The compiler treats a declared service as an additional delegated condition, which would be sound if Snort required the service and the header together. Snort 3 instead replaces the port and protocol checks by the service match, so for 2,928 rules the compiled predicate over-constrains the protocol and, for 2,877 of them, a port. This was established from the Snort 3.12.2.0 source and documentation, not by a data-plane experiment. The compiler now offers the corrected semantics as an option, but the frozen artifacts and every trace-level result in this paper use the original one.

Similarly, the 95.04% backend reduction measures P4 installation state, not traffic reduction or removal of rules from Snort. Packet-level selectivity measures candidate rates only among parser-eligible frames, whereas candidate-rule-space reduction is an offline feasibility metric and not a runtime execution reduction. Observed alert preservation is empirical and finite rather than formal equivalence. No throughput, latency, or speedup was measured. Finally, CICIDS2017 attack-window names do not by themselves establish the semantic meaning of the Snort alerts observed in those windows; an SQLi, XSS, or Brute Force window should not be interpreted as evidence that the corresponding attack class was detected unless the fired rule semantics support that conclusion.

9.2 Internal Validity

SQLi and XSS were used as development and diagnostic traces, whereas Brute Force was preserved as a held-out trace. No policy retuning was performed after inspecting the held-out trace. The strong-discriminator configuration is an operational ablation rather than a new semantic classification, and the grouping strategies were frozen before held-out validation. Experiment artifacts were frozen, with integrity checks recorded where applicable.

The causal analysis initially identified broad predicates, but the residual analysis showed that near-universal coverage persisted through distributed union coverage. This reduces the risk of attributing the observed behavior only to a small number of broad rules. It does not establish causal proof beyond the performed ablations. Flowbit dependency audits were performed for both the

original partition and the strong-discriminator partition, and no cross-partition flowbit dependency was observed in the evaluated ruleset. The absence of these static cross-partition dependencies does not establish safety for arbitrary Snort rulesets or all stateful mechanisms.

Semantic split preservation has limited discriminating power in the evaluated traces. The UNSUPPORTED partition contains 14 rules, and none of them produced an alert in any of the three traces. The comparison of the UNSUPPORTED side therefore contrasted an empty expected set with an empty run, and the candidate side necessarily coincided with the FULL alert set. The reported zero missing and zero additional alerts are correct, but a partition that assigned rules incorrectly would yield the same result whenever the misassigned rules are silent on the evaluated traffic. This measure establishes that removing the 14 UNSUPPORTED rules did not perturb the observed alerts; it does not establish that the split is correct for rules that the traces never exercised. Rulesets or traces that activate UNSUPPORTED rules would be required to give this check discriminating power. The end-to-end campaign does not close this gap, because the 14 UNSUPPORTED rules admit no synthesized witness and are outside its sample.

The campaign's witnesses are synthesized from the compiler's own IR, so the generator and the compiler could share an error. The verdict does not rest on that IR, however: a case counts only when Snort, running the original rule text, alerts on it, so an IR error that produced a wrong witness yields a VACUOUS case rather than a CONFIRMED one. Vacuous cases exposed two IR defects in `content` handling, whitespace inside the pattern and negation written as `!` outside the quotes, and both were corrected without changing any P4 artifact. The global-plan run preceded both corrections and an earlier witness-generator revision, so part of its 77 vacuous cases reflect generator and IR defects rather than the plan. The strong-discriminator run preceded only the second correction; the only sampled rule it affected, SID 28982, was recorded as vacuous. The campaign's reach is also bounded by construction. Each rule is exercised at a single point of its packet space, always on a port of its own header, so the campaign cannot observe the service-based evaluation of Section 4.5. The 1,875 rules for which no witness can be synthesized, most of which carry the richer payload and state semantics, are not tested at all, and the sample is drawn from the synthesizable subset rather than from the full ruleset.

9.3 External Validity

These results should be interpreted within the experimental Snort variable profile, one Community ruleset/version, and the selected CICIDS2017 Thursday windows. The variable profile is experimental rather than an official canonical Snort configuration. Different Community versions, custom or private rulesets, `HOME_NET` definitions, service-port profiles, rule-base sizes, traffic distributions, IDS workloads, attack mixes, or encrypted-traffic distributions may change the semantic classification, candidate rate, and candidate rule set size. The high NECESSARY rate, the global candidate rate, and the observed reductions should

therefore not be assumed to hold unchanged outside this configuration.

Only selected Thursday CICIDS2017 windows were evaluated. The dataset is old relative to current production traffic, and its traffic mix may not reflect contemporary enterprise networks. Held-out Brute Force strengthens the evidence within this experimental setup, but it does not establish broad real-world generalization to production networks, other datasets, or other IDS workloads.

The trace-based alert evidence rests on few rules: the three windows fired one, two, and five distinct SIDs. The end-to-end campaign widens rule coverage to 923 confirmed SIDs under the global plan and 851 under the strong-discriminator plan, but its traffic is synthetic, one short flow per rule, and says nothing about how often these rules fire, or which packets they fire on, in real traffic.

9.4 Implementation and Target Limitations

The prototype uses the BMv2/v1model software target rather than production ASIC hardware. It was not evaluated on production hardware, and the evaluation does not establish deployment scalability beyond the validated table state. No hardware timing, resource-utilization, line-rate, throughput, or latency measurement was conducted.

The implemented parser follows the current Ethernet/IPv4 path and does not support VLAN encapsulation. TCP, UDP, and ICMP are the principal handled L4 protocols, but the implementation does not provide generic support for every encapsulation or protocol. Parser-ineligible frames are outside the candidate-rate denominator rather than being P4 no-matches. Moreover, the P4 program marks candidate metadata; it does not implement a complete forwarding or bypass architecture that safely removes Snort from the path for arbitrary traffic.

These parser limits are handled by exclusion rather than by silent approximation. IPv4 options, fragments, truncated headers, and parser errors are gated out of predicate evaluation and marked candidate through domain fallback, so the safety relation is stated over D and the pipeline fails open elsewhere. The cost is that selectivity is claimed only over D: a deployment whose traffic contains substantially more options, fragments, or tunneling than the evaluated traces would push a larger share of packets into unconditional fallback, which is safe but less informative. Extending the parser to cover those cases selectively, rather than conservatively, remains future work.

Replay accounting is also not uniform across traces. The SQLi and XSS replays reconcile frame for frame with the data-plane counters, whereas the held-out Brute Force replay lost 87 of 1,108,658 frames (0.008%) in the transport layer, all within the candidate category. The reported held-out quantities therefore derive from the offline classifier over the complete trace, with the data-plane replay as corroboration at 99.99% frame agreement rather than as an exact reproduction.

9.5 Limitations of the Candidate-Set Analysis

The candidate rule set analysis reported mean reductions of 95.04% for SQLi, 96.11% for XSS, and 91.01% for held-out Brute Force within the P4-eligible partition, and 85.43%, 86.40%, and 81.81% over the full ruleset. These values are offline candidate-rule-space feasibility measurements, and they rest on port-constrained predicates for rules with a `service` option; compiled as Snort 3 evaluates those rules, the strong-discriminator policy bounds the full-ruleset reduction at 18.40%, and trace-level values under that compilation were not measured. The current prototype does not implement dynamic per-packet Snort rule activation, candidate-rule masks inside Snort, rule scheduling driven by P4 metadata, a candidate-set aware Snort execution engine, or throughput benchmarking of reduced rule sets.

Stateful semantics make dynamic execution a separate research problem. Even when an alert SID is present in the packet's candidate set, detection can depend on prior flow state, stream state, flowbits, temporal conditions, or state established by previous packets. The held-out 2,289/2,289 result is therefore observed alert-SID membership, not stateful dynamic-execution equivalence. The dependency audit reduces one specific class of risk but does not prove dynamic safety for all Snort state mechanisms. The campaign measured one such mechanism directly: for two HTTP rules, Snort's alert depended on a server-to-client acknowledgment that the strong-discriminator plan marks no-match, so a packet-level decision that is safe for the rule's own packets can still withhold a packet that stream inspection needs. Likewise, the grouping study recorded group structure but did not measure a grouped candidate set, and it does not show that every future grouping algorithm would fail; future evaluation should measure that set directly and consider provenance, dependencies, overlap, and target resource constraints together.

10 Conclusion

This work examined how Snort 3 detection semantics can be partially compiled into predicates executable in a P4 data plane without obscuring semantic loss, and whether those predicates remain useful when composed at ruleset scale. The result is a semantics-aware, target-aware compilation workflow in which the semantic relation of each generated predicate is explicit and predicate provenance is retained through backend lowering.

For RQ1, all 4,017 evaluated rules were parsed; 4,003 were classified NECESSARY and 14 remained UNSUPPORTED, while no rule was EXACT. Under the implementation assumptions of Section 4, the construction gives $r \Rightarrow p_r$ inside the enforced packet domain for the 1,072 NECESSARY rules whose header ports Snort checks, with options, fragments, and truncated headers retained by fail-open fallback, and without presenting partial preservation as equivalence. For the 2,928 rules with a `service` option the relation holds only where Snort evaluates the full header, because Snort 3 skips header ports and protocol once

it identifies the flow's service; the global plan still covers them through port-free entries, all but nine completely, while the strong-discriminator plan does not. For RQ2, 6,675 source predicate instances were lowered to 331 final installation entries, with a 95.04% installation-state reduction and 331/331 P4Runtime entries handled without reported encoding or validation failures on the evaluated BMv2/v1model target.

RQ3 showed that semantic coverage and operational selectivity are distinct. Under the evaluated configuration, the original binary global prefilter marked all parser-eligible packets as candidates in the SQLi, XSS, and held-out Brute Force traces; the frozen strong-discriminator ablation still marked 98.942% of parser-eligible held-out packets. Broad necessary predicates contributed initially, but the residual analysis indicated that distributed union coverage among retained predicates also sustained the near-universal candidate behavior.

For RQ4, the evaluated predicates achieved 100% observed alert preservation in the three traces, and the end-to-end campaign observed no violation of $r \Rightarrow \text{cand}$ among the sampled rules that Snort confirmed: 923 under the global plan and 851 under the strong-discriminator plan, 153 of the latter attributable to the rule's own entries alone. The same campaign showed that the global plan's broad entries leave no confirmation attributable to a single rule, and it measured two HTTP rules whose alerts depend on a packet the strong-discriminator plan marks no-match. The predicates also exposed a separate provenance-based utility: within the frozen 3,611-rule P4-eligible partition, mean candidate-rule-space reduction was 95.04% for SQLi, 96.11% for XSS, and 91.01% for held-out Brute Force, whose mean candidate set contained 324.52 rules; counting the 406 fallback rules against the full ruleset, the held-out reduction is 81.81%. This candidate-set result is an offline feasibility result, not a dynamic IDS execution result, and it depends on port-constrained predicates for service-based rules: recompiled with Snort 3's service semantics, the strong-discriminator policy keeps 739 eligible rules and bounds the full-ruleset reduction at 18.40%. The central implication is that semantic correctness and operational usefulness are distinct dimensions of IDS rule offloading: a binary candidate bit can discard rule-level information that remains useful for downstream selection. Future work should implement candidate-set-aware and dependency-aware IDS execution, investigate provenance-aware selection beyond coarse grouping, then evaluate execution cost on hardware targets and broader rulesets, profiles, traffic datasets, and parser/protocol/encapsulation coverage.

Artifact Availability

The compiler, the experimental variable profile, the analysis scripts, and the frozen output artifacts reported in this paper are available at `https://github.com/medeiros08/Snort-p4`. The repository contains the compilation pipeline from rule parsing to P4Runtime entry generation, the semantic manifest with predicate-to-rule provenance, the compiled P4 program and its P4Info, and the JSON artifacts underlying every table and figure. A regression check reproduces

Table 9: Experimental Snort variable profile used for the CICIDS2017 evaluation.

Variable	Experimental value	Role
HOME_NET	192.168.10.0/24	protected network
EXTERNAL_NET	any	external network
HTTP_SERVERS	192.168.10.0/24	HTTP server scope
SQL_SERVERS	192.168.10.0/24	SQL server scope
SMTP_SERVERS	192.168.10.0/24	SMTP server scope
TELNET_SERVERS	192.168.10.0/24	TELNET server scope
SIP_SERVERS	192.168.10.0/24	SIP server scope
HTTP_PORTS	80 8080	HTTP service ports
ORACLE_PORTS	1521	Oracle service ports
FILE_DATA_PORTS	80 110 143	file-data service ports
SIP_PORTS	5060 5061	SIP service ports
SSH_PORTS	22	SSH service ports
FTP_PORTS	21	FTP service ports

the ten frozen pipeline artifacts byte for byte from the committed sources. The end-to-end campaign's cases, per-case verdicts, and Snort alerts are under `output/campaign/`; its traces are not versioned, but the case generator at the commit that produced a run should regenerate its trace byte for byte; this was checked for the strong-discriminator run against the SHA-256 digest recorded with it.

The CICIDS2017 traces are not redistributed here; they are obtained from the Canadian Institute for Cybersecurity. The BMv2 target, Snort 3, and the P4 compiler are third-party software and are not redistributed; the exact versions used to produce the frozen artifacts are stated in Section 6, and `scripts/setup_bmv2.sh` reproduces the data-plane configuration they ran under.

A Experimental Snort Variable Profile

The evaluated CICIDS2017 configuration used the `cicids2017` profile from `config/profiles/cicids2017.json`. Its resolved variables were written to `config/snort_variables.json`. The generated Snort configuration is `config/snort3_experiment.lua`; the active-profile record identifies the same profile. Table 9 documents the resulting variables used for the 4,017-rule evaluation.

The generated configuration includes the standard Snort configuration and then explicitly disables the default inclusion of `file_magic.rules` by setting `file_inspect = {}`. Consequently, the evaluated ruleset contains only the intended 4,017 community rules.

This is an experimental variable profile, not an official canonical Snort configuration. The semantic classifications and selectivity measurements are configuration dependent: changing network variables or service-port definitions can change the generated necessary predicates and their selectivity. This profile-specific scope is considered in the threats to validity discussion.

References

[1] Ali AlSabeh, Joseph Khoury, Elie Kfoury, Jorge Crichigno, and Elias Bou-Harb. A survey on security applications of P4 programmable switches and a STRIDE-based vulnerability assessment. *Computer Networks*, 207:108800, 2022.

[2] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. P4: Programming protocol-independent packet processors. *ACM SIGCOMM Computer Communication Review*, 44(3):87–95, 2014.

[3] H. B. Brum, L. A. D. Knob, T. Ferreto, and D. Siracusa. Rethinking NIDS rule-based pre-filtering. In *IFIP/IEEE International Symposium on Integrated Network Management and Network Operations (CNSM)*, 2025.

[4] Mah-Rukh Fida and Azza H. Ahmed. Leveraging programmable data plane for network intrusion detection: A survey. *Computer Networks*, 281:112202, 2026.

[5] L. S. Hagen, A. S. Ilha, R. Parizotto, and L. P. Gaspary. RNA: Automating IDS/IPS event detection offload into programmable forwarding devices. In *IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2025.

[6] Frederik Hauser, Marco Häberle, Daniel Merling, Steffen Lindner, Vladimir Gurevich, Florian Zeiger, Reinhard Frank, and Michael Menth. A survey on data plane programming with P4: Fundamentals, advances, and applied research. *Journal of Network and Computer Applications*, 212:103561, 2023.

[7] Benjamin Lewis, Matthew Broadbent, and Nicholas Race. P4ID: P4 enhanced intrusion detection. In *2019 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, 2019.

[8] Benjamin Lewis, Matthew Broadbent, Charalampos Rotsos, and Nicholas Race. 4MIDable: Flexible network offloading for security VNFs. *Journal of Network and Systems Management*, 31:52, 2023.

[9] Ali Mazloum, Ali AlSabeh, Elie Kfoury, and Jorge Crichigno. Security applications in P4: Implementation and lessons learned. *Computer Networks*, 257:111011, 2025.

[10] p4lang. BMv2: The behavioral model software switch. `https://github.com/p4lang/behavioral-model`, 2025. `simple_switch_grpc` 1.15.0, commit 68f4a978.

[11] Martin Roesch. Snort: Lightweight intrusion detection for networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*, pages 229–238, Seattle, WA, USA, 1999. USENIX Association.

[12] Eduardo Vudala Senoski, João Ribeiro Andreotii, Raphael Kaviak Machnicki, Jorge Pires Correia, and Vinicius Fulber-Garcia. P4NIPS: P4 network intrusion prevention system. In *Anais Estendidos do Simpósio Brasileiro de Cibersegurança (SBSeg)*, pages 75–84, 2025.

[13] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018.

[14] Kairo Tavares and Tiago Ferreto. P4-ONIDS: A P4-based NIDS optimized for constrained programmable data planes in SDN. In *Anais do XXXIX Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos*, pages 434–447, 2021.

[15] The P4 Language Consortium. *$P4_{16}$ Language Specification*, 2023. Version 1.2.4, `https://p4.org/specs/`.

[16] The P4.org API Working Group. *P4Runtime Specification*, 2024. Version 1.4.1, `https://p4.org/specs/`.

[17] The Snort Team. *Snort 3 User Manual*, 2026. Snort 3.12.2.0, `https://github.com/snort3/snort3`.